

Windows Information Protection (WIP) developer guide

Article • 10/27/2022

An *enlightened* app differentiates between corporate and personal data and knows which to protect based on Windows Information Protection (WIP) policies defined by the administrator.

In this guide, we'll show you how to build one. When you're done, policy administrators will be able to trust your app to consume their organization's data. And employees will love that you've kept their personal data intact on their device even if they un-enroll from the organization's mobile device management (MDM) or leave the organization entirely.

Note This guide helps you enlighten a UWP app. If you want to enlighten a C++ Windows desktop app, see [Windows Information Protection \(WIP\) developer guide \(C++\)](#).

You can read more about WIP and enlightened apps here: [Windows Information Protection \(WIP\)](#).

You can find a complete sample [here](#) [↗](#).

If you're ready to go through each task, let's start.

First, gather what you need

You'll need these:

- A test Virtual Machine (VM) that runs Windows 10, version 1607 or later. You'll debug your app against this test VM.
- A development computer that runs Windows 10, version 1607 or later. This could be your test VM if you have Visual Studio installed on it.

Setup your development environment

You'll do these things:

- [Install the WIP Setup Developer Assistant onto your test VM](#)

- Create a protection policy by using the WIP Setup Developer Assistant
- Setup a Visual Studio project
- Setup remote debugging
- Add namespaces to your code files

Install the WIP Setup Developer Assistant onto your test VM

Use this tool to setup a Windows Information Protection policy on your test VM.

Download the tool here: [WIP Setup Developer Assistant](#).

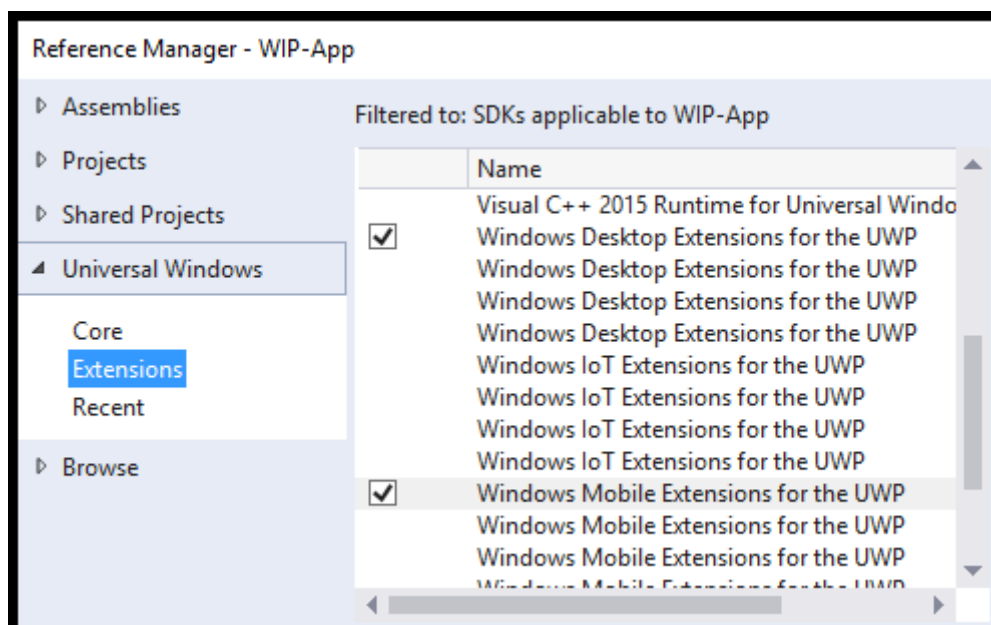
Create a protection policy

Define your policy by adding information to each section in the WIP setup developer assistant. Choose the help icon next to any setting to learn more about how to use it.

For more general guidance about how to use this tool, see the Version notes section on the app download page.

Setup a Visual Studio project

1. On your development computer, open your project.
2. Add a reference to the desktop and mobile extensions for Universal Windows Platform (UWP).



3. Add this capability to your package manifest file:

XML

```
<rescap:Capability Name="enterpriseDataPolicy"/>
```

Optional Reading: The "rescap" prefix means *Restricted Capability*. See [Special and restricted capabilities](#).

4. Add this namespace to your package manifest file:

XML

```
xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
```

5. Add the namespace prefix to the `<ignorableNamespaces>` element of your package manifest file.

XML

```
<IgnorableNamespaces="uap mp rescap">
```

This way, if your app runs on a version of the Windows operating system that doesn't support restricted capabilities, Windows will ignore the `enterpriseDataPolicy` capability.

Setup remote debugging

Install Visual Studio Remote Tools on your test VM only if you are developing your app on a computer other than your VM. Then, on your development computer start the remote debugger and see if your app runs on the test VM.

See [Remote PC instructions](#).

Add these namespaces to your code files

Add these using statements to the top of your code files(The snippets in this guide use them):

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Windows.Security.EnterpriseData;
using Windows.Web.Http;
using Windows.Storage.Streams;
using Windows.ApplicationModel.DataTransfer;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml;
using Windows.ApplicationModel.Activation;
using Windows.Web.Http.Filters;
using Windows.Storage;
using Windows.Data.Xml.Dom;
using Windows.Foundation.Metadata;
using Windows.Web.Http.Headers;
```

Determine whether to use WIP APIs in your app

Ensure that the operating system that runs your app supports WIP and that WIP is enabled on the device.

C#

```
bool use_WIP_APIS = false;

if ((ApiInformation.IsApiContractPresent
    ("Windows.Security.EnterpriseData.EnterpriseDataContract", 3)
    && ProtectionPolicyManager.IsProtectionEnabled))
{
    use_WIP_APIS = true;
}
else
{
    use_WIP_APIS = false;
}
```

Don't call WIP APIs if the operating system doesn't support WIP or WIP is not enabled on the device.

Read enterprise data

To read protected files, network endpoints, clipboard data and data that you accept from a Share contract, your app will have to request access.

Windows Information Protection gives your app permission if your app is on the protection policy's allowed list.

In this section:

- Read data from a file
- Read data from a network endpoint
- Read data from the clipboard
- Read data from a Share contract

Read data from a file

Step 1: Get the file handle

C#

```
Windows.Storage.StorageFolder storageFolder =  
    Windows.Storage.ApplicationData.Current.LocalFolder;  
  
Windows.Storage.StorageFile file =  
    await storageFolder.GetFilesAsync(fileName);
```

Step 2: Determine whether your app can open the file

Call `FileProtectionManager.GetProtectionInfoAsync` to determine whether your app can open the file.

C#

```
FileProtectionInfo protectionInfo = await  
FileProtectionManager.GetProtectionInfoAsync(file);  
  
if ((protectionInfo.Status != FileProtectionStatus.Protected &&  
    protectionInfo.Status != FileProtectionStatus.Unprotected))  
{  
    return false;  
}  
else if (protectionInfo.Status == FileProtectionStatus.Revoked)  
{  
    // Code goes here to handle this situation. Perhaps, show UI  
    // saying that the user's data has been revoked.  
}
```

A `FileProtectionStatus` value of **Protected** means that the file is protected and your app can open it because your app is on the policy's allowed list.

A [FileProtectionStatus](#) value of **UnProtected** means that the file is not protected and your app can open the file even your app is not on the policy's allowed list.

APIs

[FileProtectionManager.GetProtectionInfoAsync](#)

[FileProtectionInfo](#)

[FileProtectionStatus](#)

[ProtectionPolicyManager.IsIdentityManaged](#)

Step 3: Read the file into a stream or buffer

Read the file into a stream

C#

```
var stream = await file.OpenAsync(Windows.Storage.FileAccessMode.ReadWrite);
```

Read the file into a buffer

C#

```
var buffer = await Windows.Storage.FileIO.ReadBufferAsync(file);
```

Read data from a network endpoint

Create a protected thread context to read from an enterprise endpoint.

Step 1: Get the identity of the network endpoint

C#

```
Uri resourceURI = new Uri("http://contoso.com/stockData.xml");

Windows.Networking.HostName hostName =
    new Windows.Networking.HostName(resourceURI.Host);

string identity = await ProtectionPolicyManager.
    GetPrimaryManagedIdentityForNetworkEndpointAsync(hostName);
```

If the endpoint isn't managed by policy, you'll get back an empty string.

APIs

[ProtectionPolicyManager.GetPrimaryManagedIdentityForNetworkEndpointAsync](#)

Step 2: Create a protected thread context

If the endpoint is managed by policy, create a protected thread context. This tags any network connections that you make on the same thread to the identity.

It also gives you access to enterprise network resources that are managed by that policy.

C#

```
if (!string.IsNullOrEmpty(identity))
{
    using (ThreadNetworkContext threadNetworkContext =
        ProtectionPolicyManager.CreateCurrentThreadNetworkContext(identity))
    {
        return await GetDataFromNetworkRedirectHelperMethod(resourceURI);
    }
}
else
{
    return await GetDataFromNetworkRedirectHelperMethod(resourceURI);
}
```

This example encloses socket calls in a `using` block. If you don't do this, make sure that you close the thread context after you've retrieved your resource. See [ThreadNetworkContext.Close](#).

Don't create any personal files on that protected thread because those files will be automatically encrypted.

The [ProtectionPolicyManager.CreateCurrentThreadNetworkContext](#) method returns a [ThreadNetworkContext](#) object whether or not the endpoint is being managed by policy. If your app handles both personal and enterprise resources, call [ProtectionPolicyManager.CreateCurrentThreadNetworkContext](#) for all identities. After you get the resource, dispose the `ThreadNetworkContext` to clear any identity tag from the current thread.

APIs

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

[ProtectionPolicyManager.CreateCurrentThreadNetworkContext](#)

Step 3: Read the resource into a buffer

C#

```
private static async Task<IBuffer> GetDataFromNetworkHelperMethod(Uri
resourceURI)
{
    HttpClient client;

    client = new HttpClient();

    try { return await client.GetBufferAsync(resourceURI); }

    catch (Exception) { return null; }
}
```

(Optional) Use a header token instead of creating a protected thread context

C#

```
public static async Task<IBuffer> GetDataFromNetworkbyUsingHeader(Uri
resourceURI)
{
    HttpClient client;

    Windows.Networking.HostName hostName =
        new Windows.Networking.HostName(resourceURI.Host);

    string identity = await ProtectionPolicyManager.
        GetPrimaryManagedIdentityForNetworkEndpointAsync(hostName);

    if (!string.IsNullOrEmpty(identity))
    {
        client = new HttpClient();

        HttpRequestHeaderCollection headerCollection =
client.DefaultRequestHeaders;

        headerCollection.Add("X-MS-Windows-HttpClient-EnterpriseId",
identity);

        return await GetDataFromNetworkbyUsingHeaderHelperMethod(client,
resourceURI);
    }
    else
    {
        client = new HttpClient();
        return await GetDataFromNetworkbyUsingHeaderHelperMethod(client,
resourceURI);
    }
}

private static async Task<IBuffer>
GetDataFromNetworkbyUsingHeaderHelperMethod(HttpClient client, Uri
resourceURI)
```



```
{
    try { return await client.GetBufferAsync(resourceURI); }
    catch (Exception) { return null; }
}
```

Handle page redirects

Sometimes a web server will redirect traffic to a more current version of a resource.

To handle this, make requests until the response status of your request has a value of **OK**.

Then use the URI of that response to get the identity of the endpoint. Here's one way to do this:

C#

```
private static async Task<IBuffer>
GetDataFromNetworkRedirectHelperMethod(Uri resourceURI)
{
    HttpClient client = null;

    HttpBaseProtocolFilter filter = new HttpBaseProtocolFilter();
    filter.AllowAutoRedirect = false;

    client = new HttpClient(filter);

    HttpResponseMessage response = null;

    HttpRequestMessage message = new HttpRequestMessage(HttpMethod.Get,
resourceURI);
    response = await client.SendRequestAsync(message);

    if (response.StatusCode == HttpStatusCode.MultipleChoices ||
        response.StatusCode == HttpStatusCode.MovedPermanently ||
        response.StatusCode == HttpStatusCode.Found ||
        response.StatusCode == HttpStatusCode.SeeOther ||
        response.StatusCode == HttpStatusCode.NotModified ||
        response.StatusCode == HttpStatusCode.UseProxy ||
        response.StatusCode == HttpStatusCode.TemporaryRedirect ||
        response.StatusCode == HttpStatusCode.PermanentRedirect)
    {
        message = new HttpRequestMessage(HttpMethod.Get,
message.RequestUri);
        response = await client.SendRequestAsync(message);

        try { return await response.Content.ReadAsBufferAsync(); }

        catch (Exception) { return null; }
    }
}
```

```

else
{
    try { return await response.Content.ReadAsBufferAsync(); }

    catch (Exception) { return null; }
}
}

```

APIs

[ProtectionPolicyManager.GetPrimaryManagedIdentityForNetworkEndpointAsync](#)

[ProtectionPolicyManager.CreateCurrentThreadNetworkContext](#)

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

Read data from the clipboard

Get permission to use data from the clipboard

To get data from the clipboard, ask Windows for permission. Use [DataPackageView.RequestAccessAsync](#) to do that.

C#

```

public static async Task PasteText(TextBox textBox)
{
    DataPackageView dataPackageView = Clipboard.GetContent();

    if (dataPackageView.Contains(StandardDataFormats.Text))
    {
        ProtectionPolicyEvaluationResult result = await
dataPackageView.RequestAccessAsync();

        if (result == ProtectionPolicyEvaluationResult..Allowed)
        {
            string contentsOfClipboard = await
dataPackageView.GetTextAsync();
            textBox.Text = contentsOfClipboard;
        }
    }
}

```

APIs

[DataPackageView.RequestAccessAsync](#)

Hide or disable features that use clipboard data

Determine whether current view has permission to get data that is on the clipboard.

If it doesn't, you can disable or hide controls that let users paste information from the clipboard or preview its contents.

C#

```
private bool IsClipboardAllowedAsync()
{
    ProtectionPolicyEvaluationResult protectionPolicyEvaluationResult =
    ProtectionPolicyEvaluationResult.Blocked;

    DataPackageView dataPackageView = Clipboard.GetContent();

    if (dataPackageView.Contains(StandardDataFormats.Text))

        protectionPolicyEvaluationResult =

ProtectionPolicyManager.CheckAccess(dataPackageView.Properties.EnterpriseId,
    ProtectionPolicyManager.GetForCurrentView().Identity);

    return (protectionPolicyEvaluationResult ==
    ProtectionPolicyEvaluationResult.Allowed |
        protectionPolicyEvaluationResult ==
    ProtectionPolicyEvaluationResult.ConsentRequired);
}
```

APIs

[ProtectionPolicyEvaluationResult](#)

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

Prevent users from being prompted with a consent dialog box

A new document isn't *personal* or *enterprise*. It's just new. If a user pastes enterprise data into it, Windows enforces policy and the user is prompted with a consent dialog. This code prevents that from happening. This task is not about helping to protect data. It's more about keeping users from receiving the consent dialog box in cases where your app creates a brand new item.

C#

```
private async void PasteText(bool isNewEmptyDocument)
{
    DataPackageView dataPackageView = Clipboard.GetContent();

    if (dataPackageView.Contains(StandardDataFormats.Text))
    {
```

```

        if (!string.IsNullOrEmpty(dataPackageView.Properties.EnterpriseId))
        {
            if (isNewEmptyDocument)
            {
                ProtectionPolicyManager.TryApplyProcessUIPolicy(dataPackageView.Properties.EnterpriseId);
                string contentsOfClipboard = contentsOfClipboard = await
                    dataPackageView.GetTextAsync();
                // add this string to the new item or document here.

            }
            else
            {
                ProtectionPolicyEvaluationResult result = await
                    dataPackageView.RequestAccessAsync();

                if (result == ProtectionPolicyEvaluationResult.Allowed)
                {
                    string contentsOfClipboard = contentsOfClipboard = await
                        dataPackageView.GetTextAsync();
                    // add this string to the new item or document here.
                }
            }
        }
    }
}

```

APIs

[DataPackageView.RequestAccessAsync](#)

[ProtectionPolicyEvaluationResult](#)

[ProtectionPolicyManager.TryApplyProcessUIPolicy](#)

Read data from a Share contract

When employees choose your app to share their information, your app will open a new item that contains that content.

As we mentioned earlier, a new item isn't *personal* or *enterprise*. It's just new. If your code adds enterprise content to the item, Windows enforces policy and the user is prompted with a consent dialog. This code prevents that from happening.

C#

```

protected override async void
OnShareTargetActivated(ShareTargetActivatedEventArgs args)
{
    bool isNewEmptyDocument = true;
    string identity = "corp.microsoft.com";
}

```

```

    ShareOperation shareOperation = args.ShareOperation;
    if (shareOperation.Data.Contains(StandardDataFormats.Text))
    {
        if
(!string.IsNullOrEmpty(shareOperation.Data.Properties.EnterpriseId))
        {
            if (isNewEmptyDocument)
                // If this is a new and empty document, and we're allowed to
access
                // the data, then we can avoid popping the consent dialog

ProtectionPolicyManager.TryApplyProcessUIPolicy(shareOperation.Data Properti
es.EnterpriseId);
            else
            {
                // In this case, we can't optimize the workflow, so we just
                // request consent from the user in this case.

                ProtectionPolicyEvaluationResult
protectionPolicyEvaluationResult = await
shareOperation.Data.RequestAccessAsync();

                if (protectionPolicyEvaluationResult ==
ProtectionPolicyEvaluationResult.Allowed)
                {
                    string text = await shareOperation.Data.GetTextAsync();

                    // Do something with that text.
                }
            }
        }
        else
        {
            // If the data has no enterprise identity, then we already have
access.
            string text = await shareOperation.Data.GetTextAsync();

            // Do something with that text.
        }
    }
}

```

APIs

ProtectionPolicyManager.RequestAccessAsync

ProtectionPolicyEvaluationResult

ProtectionPolicyManager.TryApplyProcessUIPolicy

Protect enterprise data

Protect enterprise data that leaves your app. Data leaves your app when you show it in a page, save it to a file or network endpoint, or through a share contract.

In this section:

- [Protect data that appears in pages](#)
- [Protect data to a file as a background process](#)
- [Protect part of a file](#)
- [Read the protected part of a file](#)
- [Protect data to a folder](#)
- [Protect data to a network end point](#)
- [Protect data that your app shares through a share contract](#)
- [Protect files that you copy to another location](#)
- [Protect enterprise data when the screen of the device is locked](#)

Protect data that appears in pages

When you show data in a page, let Windows know what type of data it is (personal or enterprise). To do that, *tag* the current app view or tag the entire app process.

When you tag the view or the process, Windows enforces policy on it. This helps prevent data leaks that result from actions that your app doesn't control. For example, on a computer, a user could use CTRL-V to copy enterprise information from a view and then paste that information to another app. Windows protects against that. Windows also helps to enforce share contracts.

Tag the current app view

Do this if your app has multiple views where some views consume enterprise data and some consume personal data.

C#

```
// tag as enterprise data. "identity" the string that contains the
enterprise ID.
// You'd get that from a file, network endpoint, or clipboard data package.
ProtectionPolicyManager.GetForCurrentView().Identity = identity;

// tag as personal data.
ProtectionPolicyManager.GetForCurrentView().Identity = String.Empty;
```

APIs

[ProtectionPolicyManager.GetForCurrentView](#)

Tag the process

Do this if all views in your app will work with only one type of data (personal or enterprise).

This prevents you from having to manage independently tagged views.

C#

```
// tag as enterprise data. "identity" the string that contains the
enterprise ID.
// You'd get that from a file, network endpoint, or clipboard data package.
bool result =
    ProtectionPolicyManager.TryApplyProcessUIPolicy(identity);

// tag as personal data.
ProtectionPolicyManager.ClearProcessUIPolicy();
```

APIs

ProtectionPolicyManager.TryApplyProcessUIPolicy

Protect data to a file

Create a protected file and then write to it.

Step 1: Determine if your app can create an enterprise file

Your app can create an enterprise file if the identity string is managed by policy and your app is on the Allowed list of that policy.

C#

```
if (!ProtectionPolicyManager.IsIdentityManaged(identity)) return false;
```

APIs

ProtectionPolicyManager.IsIdentityManaged

Step 2: Create the file and protect it to the identity

C#

```
StorageFolder storageFolder = ApplicationData.Current.LocalFolder;
StorageFile storageFile = await storageFolder.CreateFileAsync("sample.txt",
    CreationCollisionOption.ReplaceExisting);

FileProtectionInfo fileProtectionInfo =
    await FileProtectionManager.ProtectAsync(storageFile, identity);
```

APIs

[FileProtectionManager.ProtectAsync](#)

Step 3: Write that stream or buffer to the file

Write a stream

C#

```
if (fileProtectionInfo.Status == FileProtectionStatus.Protected)
{
    var stream = await storageFile.OpenAsync(FileAccessMode.ReadWrite);

    using (var outputStream = stream.GetOutputStreamAt(0))
    {
        using (var dataWriter = new DataWriter(outputStream))
        {
            dataWriter.WriteString(enterpriseData);
        }
    }
}
```

Write a buffer

C#

```
if (fileProtectionInfo.Status == FileProtectionStatus.Protected)
{
    var buffer =
        Windows.Security.Cryptography.CryptographicBuffer.ConvertStringToBinary(
            enterpriseData,
            Windows.Security.Cryptography.BinaryStringEncoding.Utf8);

    await FileIO.WriteBufferAsync(storageFile, buffer);
}
```

APIs

[FileProtectionInfo](#)

Protect data to a file as a background process

This code can run while the screen of the device is locked. If the administrator configured a secure "Data protection under lock" (DPL) policy, Windows removes the encryption keys required to access protected resources from device memory. This prevents data leaks if the device is lost. This same feature also removes keys associated with protected files when their handles are closed.

You'll have to use an approach that keeps the file handle open when you create a file.

Step 1: Determine if you can create an enterprise file

You can create an enterprise file if the identity that you're using is managed by policy and your app is on the allowed list of that policy.

C#

```
if (!ProtectionPolicyManager.IsIdentityManaged(identity)) return false;
```

APIs

[ProtectionPolicyManager.IsIdentityManaged](#)

Step 2: Create a file and protect it to the identity

The [FileProtectionManager.CreateProtectedAndOpenAsync](#) creates a protected file and keeps the file handle open while you write to it.

C#

```
StorageFolder storageFolder = ApplicationData.Current.LocalFolder;

ProtectedFileCreateResult protectedFileCreateResult =
    await FileProtectionManager.CreateProtectedAndOpenAsync(storageFolder,
        "sample.txt", identity, CreationCollisionOption.ReplaceExisting);
```

APIs

[FileProtectionManager.CreateProtectedAndOpenAsync](#)

Step 3: Write a stream or buffer to the file

This example writes a stream to a file.

C#

```
if (protectedFileCreateResult.ProtectionInfo.Status ==
FileProtectionStatus.Protected)
{
    IOOutputStream outputStream =
        protectedFileCreateResult.Stream.GetOutputStreamAt(0);

    using (DataWriter writer = new DataWriter(outputStream))
    {
        writer.WriteString(enterpriseData);
        await writer.StoreAsync();
        await writer.FlushAsync();
    }

    outputStream.Dispose();
}
else if (protectedFileCreateResult.ProtectionInfo.Status ==
FileProtectionStatus.AccessSuspended)
{
    // Perform any special processing for the access suspended case.
}
```

APIs

[ProtectedFileCreateResult.ProtectionInfo](#)

[FileProtectionStatus](#)

[ProtectedFileCreateResult.Stream](#)

Protect part of a file

In most cases, it's cleaner to store enterprise and personal data separately but you can store them to the same file if you want. For example, Microsoft Outlook can store enterprise mails alongside of personal mails in a single archive file.

Encrypt the enterprise data but not the entire file. That way, users can continue using that file even if they un-enroll from MDM or their enterprise data access rights are revoked. Also, your app should keep track of what data it encrypts so that it knows what data to protect when it reads the file back into memory.

Step 1: Add enterprise data to an encrypted stream or buffer

C#

```
string enterpriseDataString = "<employees><employee><name>Bill</name><social>xxx-xxx-xxxx</social></employee></employees>";

var enterpriseData=
```

```
Windows.Security.Cryptography.CryptographicBuffer.ConvertStringToBinary(
    enterpriseDataString,
    Windows.Security.Cryptography.BinaryStringEncoding.Utf8);

BufferProtectUnprotectResult result =
    await DataProtectionManager.ProtectAsync(enterpriseData, identity);

enterpriseData= result.Buffer;
```

APIs

DataProtectionManager.ProtectAsync

BufferProtectUnprotectResult.buffer

Step 2: Add personal data to an unencrypted stream or buffer

C#

```
string personalDataString = "<recipies><recipe><name>BillsCupCakes</name><br><cooktime>30</cooktime></recipe></recipies>";

var personalData =
    Windows.Security.Cryptography.CryptographicBuffer.ConvertStringToBinary(
        personalDataString,
        Windows.Security.Cryptography.BinaryStringEncoding.Utf8);
```

Step 3: Write both streams or buffers to a file

C#

```
StorageFolder storageFolder = ApplicationData.Current.LocalFolder;

StorageFile storageFile = await storageFolder.CreateFileAsync("data.xml",
    CreationCollisionOption.ReplaceExisting);

// Write both buffers to the file and save the file.

var stream = await storageFile.OpenAsync(FileAccessMode.ReadWrite);

using (var outputStream = stream.GetOutputStreamAt(0))
{
    using (var dataWriter = new DataWriter(outputStream))
    {
        dataWriter.WriteBuffer(enterpriseData);
        dataWriter.WriteBuffer(personalData);

        await dataWriter.StoreAsync();
        await outputStream.FlushAsync();
    }
}
```

Step 4: Keep track of the location of your enterprise data in the file

It's the responsibility of your app to keep track of the data in that file that is enterprise owned.

You can store that information in a property associated with the file, in a database, or in some header text in the file.

This example, saves that information to a separate XML file.

C#

```
StorageFile metaDataFile = await
storageFolder.CreateFileAsync("metadata.xml",
    CreationCollisionOption.ReplaceExisting);

await Windows.Storage.FileIO.WriteTextAsync
    (metaDataFile, "<EnterpriseDataMarker start='0' end='" +
enterpriseData.Length.ToString() +
    "'></EnterpriseDataMarker>");
```

Read the protected part of a file

Here's how you'd read the enterprise data out of that file.

Step 1: Get the position of your enterprise data in the file

C#

```
Windows.Storage.StorageFolder storageFolder =
    Windows.Storage.ApplicationData.Current.LocalFolder;

Windows.Storage.StorageFile metaDataFile =
    await storageFolder.GetFileAsync("metadata.xml");

string metaData = await Windows.Storage.FileIO.ReadTextAsync(metaDataFile);

XmlDocument doc = new XmlDocument();

doc.LoadXml(metaData);

uint startPosition =

Convert.ToUInt16((doc.FirstChild.Attributes.GetNamedItem("start")).InnerText
);

uint endPosition =

Convert.ToUInt16((doc.FirstChild.Attributes.GetNamedItem("end")).InnerText);
```

Step 2: Open the data file and make sure that it's not protected

C#

```
Windows.Storage.StorageFile dataFile =  
    await storageFolder.GetFilesAsync("data.xml");  
  
FileProtectionInfo protectionInfo =  
    await FileProtectionManager.GetProtectionInfoAsync(dataFile);  
  
if (protectionInfo.Status == FileProtectionStatus.Protected)  
    return false;
```

APIs

[FileProtectionManager.GetProtectionInfoAsync](#)

[FileProtectionInfo](#)

[FileProtectionStatus](#)

Step 3: Read the enterprise data from the file

C#

```
var stream = await  
dataFile.OpenAsync(Windows.Storage.FileAccessMode.ReadWrite);  
  
stream.Seek(startPosition);  
  
Windows.Storage.Streams.Buffer tempBuffer = new  
Windows.Storage.Streams.Buffer(50000);  
  
IBuffer enterpriseData = await stream.ReadAsync(tempBuffer, endPosition,  
InputStreamOptions.None);
```

Step 4: Decrypt the buffer that contains enterprise data

C#

```
DataProtectionInfo dataProtectionInfo =  
    await DataProtectionManager.GetProtectionInfoAsync(enterpriseData);  
  
if (dataProtectionInfo.Status == DataProtectionStatus.Protected)  
{  
    BufferProtectUnprotectResult result = await  
DataProtectionManager.UnprotectAsync(enterpriseData);  
    enterpriseData = result.Buffer;  
}  
else if (dataProtectionInfo.Status == DataProtectionStatus.Revoked)  
{  
    // Code goes here to handle this situation. Perhaps, show UI
```

```
    // saying that the user's data has been revoked.  
}
```

APIs

DataProtectionInfo

DataProtectionManager.GetProtectionInfoAsync

Protect data to a folder

You can create a folder and protect it. That way any items that you add to that folder are automatically protected.

C#

```
private async Task<bool> CreateANewFolderAndProtectItAsync(string  
folderName, string identity)  
{  
    if (!ProtectionPolicyManager.IsIdentityManaged(identity)) return false;  
  
    StorageFolder storageFolder = ApplicationData.Current.LocalFolder;  
    StorageFolder newStorageFolder =  
        await storageFolder.CreateFolderAsync(folderName);  
  
    FileProtectionInfo fileProtectionInfo =  
        await FileProtectionManager.ProtectAsync(newStorageFolder,  
identity);  
  
    if (fileProtectionInfo.Status != FileProtectionStatus.Protected)  
    {  
        // Protection failed.  
        return false;  
    }  
    return true;  
}
```

Make sure that the folder is empty before you protect it. You can't protect a folder that already contains items.

APIs

ProtectionPolicyManager.IsIdentityManaged

FileProtectionManager.ProtectAsync

FileProtectionInfo.Identity

FileProtectionInfo.Status

Protect data to a network end point

Create a protected thread context to send that data to an enterprise endpoint.

Step 1: Get the identity of the network endpoint

C#

```
Windows.Networking.HostName hostName =  
    new Windows.Networking.HostName(resourceURI.Host);  
  
string identity = await ProtectionPolicyManager.  
    GetPrimaryManagedIdentityForNetworkEndpointAsync(hostName);
```

APIs

[ProtectionPolicyManager.GetPrimaryManagedIdentityForNetworkEndpointAsync](#)

Step 2: Create a protected thread context and send data to the network endpoint

C#

```
HttpClient client = null;  
  
if (!string.IsNullOrEmpty(m_EnterpriseId))  
{  
    ProtectionPolicyManager.GetForCurrentView().Identity = identity;  
  
    using (ThreadNetworkContext threadNetworkContext =  
ProtectionPolicyManager.CreateCurrentThreadNetworkContext(identity))  
    {  
        client = new HttpClient();  
        HttpRequestMessage message = new HttpRequestMessage(HttpMethod.Put,  
resourceURI);  
        message.Content = new HttpStreamContent(dataToWrite);  
  
        HttpResponseMessage response = await  
client.SendRequestAsync(message);  
  
        if (response.StatusCode == HttpStatusCode.Ok)  
            return true;  
        else  
            return false;  
    }  
}  
else  
{  
    return false;  
}
```

APIs

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

[ProtectionPolicyManager.CreateCurrentThreadNetworkContext](#)

Protect data that your app shares through a share contract

If you want users to share content from your app, you'll have to implement a share contract and handle the [DataTransferManager.DataRequested](#) event.

In your event handler, set the enterprise identity context in the data package.

C#

```
private void OnShareSourceOperation(object sender, RoutedEventArgs e)
{
    // Register the current page as a share source (or you could do this
    // earlier in your app).
    DataTransferManager.GetForCurrentView().DataRequested +=
    OnDataRequested;
    DataTransferManager.ShowShareUI();
}

private void OnDataRequested(DataTransferManager sender,
DataRequestedEventArgs args)
{
    if (!string.IsNullOrEmpty(this.shareSourceContent))
    {
        var protectionPolicyManager =
        ProtectionPolicyManager.GetForCurrentView();
        DataPackage requestData = args.Request.Data;
        requestData.Properties.Title = this.shareSourceTitle;
        requestData.Properties.EnterpriseId =
        protectionPolicyManager.Identity;
        requestData.SetText(this.shareSourceContent);
    }
}
```

APIs

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

Protect files that you copy to another location

C#


```
private async void CopyProtectionFromOneFileToAnother
    (StorageFile sourceStorageFile, StorageFile targetStorageFile)
{
    bool copyResult = await
        FileProtectionManager.CopyProtectionAsync(sourceStorageFile,
        targetStorageFile);

    if (!copyResult)
    {
        // Copying failed. To diagnose, you could check the file's status.
        // (call FileProtectionManager.GetProtectionInfoAsync and
        // check FileProtectionInfo.Status).
    }
}
```

APIs

`FileProtectionManager.CopyProtectionAsync`

Protect enterprise data when the screen of the device is locked

Remove all sensitive data in memory when the device is locked. When the user unlocks the device, your app can safely add that data back.

Handle the `ProtectionPolicyManager.ProtectedAccessSuspending` event so that your app knows when the screen is locked. This event is raised only if the administrator configures a secure data protection under lock policy. Windows temporarily removes the data protection keys that are provisioned on the device. Windows removes these keys to ensure that there is no unauthorized access to encrypted data while the device is locked and possibly not in possession of its owner.

Handle the `ProtectionPolicyManager.ProtectedAccessResumed` event so that your app knows when the screen is unlocked. This event is raised regardless of whether the administrator configures a secure data protection under lock policy.

Remove sensitive data in memory when the screen is locked

Protect sensitive data, and close any file streams that your app has opened on protected files to help ensure that the system doesn't cache any sensitive data in memory.

This example saves content from a textblock to an encrypted buffer and removes the content from that textblock.

```

private async void ProtectionPolicyManager_ProtectedAccessSuspending(object
sender, ProtectedAccessSuspendingEventArgs e)
{
    Deferral deferral = e.GetDeferral();

    if (ProtectionPolicyManager.GetForCurrentView().Identity !=
String.Empty)
    {
        IBuffer documentBodyBuffer =
CryptographicBuffer.ConvertStringToBinary
(documentTextBlock.Text, BinaryStringEncoding.Utf8);

        BufferProtectUnprotectResult result = await
DataProtectionManager.ProtectAsync
(documentBodyBuffer,
ProtectionPolicyManager.GetForCurrentView().Identity);

        if (result.ProtectionInfo.Status == DataProtectionStatus.Protected)
        {
            this.protectedDocumentBuffer = result.Buffer;
            documentTextBlock.Text = null;
        }
    }

    // Close any open streams that you are actively working with
    // to make sure that we have no unprotected content in memory.

    // Optionally, code goes here to use e.Deadline to determine whether we
have more
    // than 15 seconds left before the suspension deadline. If we do then
process any
    // messages queued up for sending while we are still able to access
them.

    deferral.Complete();
}

```

APIs

[ProtectionPolicyManager.ProtectedAccessSuspending](#)

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

[DataProtectionManager.ProtectAsync](#)

[BufferProtectUnprotectResult.buffer](#)

[ProtectedAccessSuspendingEventArgs.GetDeferral](#)

[Deferral.Complete](#)

Add back sensitive data when the device is unlocked

[ProtectionPolicyManager.ProtectedAccessResumed](#) is raised when the device is unlocked and the keys are available on the device again.

[ProtectedAccessResumedEventArgs.Identities](#) is an empty collection if the administrator hasn't configured a secure data protection under lock policy.

This example does the reverse of the previous example. It decrypts the buffer, adds information from that buffer back to the textbox and then disposes of the buffer.

C#

```
private async void ProtectionPolicyManager_ProtectedAccessResumed(object sender, ProtectedAccessResumedEventArgs e)
{
    if (ProtectionPolicyManager.GetForCurrentView().Identity != String.Empty)
    {
        BufferProtectUnprotectResult result = await DataProtectionManager.UnprotectAsync(this.protectedDocumentBuffer);

        if (result.ProtectionInfo.Status == DataProtectionStatus.Unprotected)
        {
            // Restore the unprotected version.
            documentTextBlock.Text = CryptographicBuffer.ConvertBinaryToString(BinaryStringEncoding.Utf8, result.Buffer);
            this.protectedDocumentBuffer = null;
        }
    }
}
```

APIs

[ProtectionPolicyManager.ProtectedAccessResumed](#)

[ProtectionPolicyManager.GetForCurrentView](#)

[ProtectionPolicyManager.Identity](#)

[DataProtectionManager.UnprotectAsync](#)

[BufferProtectUnprotectResult.Status](#)

Handle enterprise data when protected content is revoked

If you want your app to be notified when the device is un-enrolled from MDM or when the policy administrator explicitly revokes access to enterprise data, handle the

[ProtectionPolicyManager_ProtectedContentRevoked](#) event.

This example determines if the data in an enterprise mailbox for an email app has been revoked.

C#

```
private string mailIdentity = "contoso.com";

void MailAppSetup()
{
    ProtectionPolicyManager.ProtectedContentRevoked +=
    ProtectionPolicyManager_ProtectedContentRevoked;
    // Code goes here to set up mailbox for 'mailIdentity'.
}

private void ProtectionPolicyManager_ProtectedContentRevoked(object sender,
ProtectedContentRevokedEventArgs e)
{
    if (!new System.Collections.Generic.List<string>(e.Identities).Contains
(this.mailIdentity))
    {
        // This event is not for our identity.
        return;
    }

    // Code goes here to delete any metadata associated with 'mailIdentity'.
}
```

APIs

[ProtectionPolicyManager_ProtectedContentRevoked](#)

Related topics

[Windows Information Protection \(WIP\) sample](#) 

Enterprise Shared Storage

Article • 10/20/2022

The shared storage consists of two locations, where apps with the restricted capability **enterpriseDeviceLockdown** and an Enterprise certificate have full read and write access. Note that the **enterpriseDeviceLockdown** capability allows apps to use the device lock down API and access the enterprise shared storage folders. For more information about the API, see [Windows.Embedded.DeviceLockdown](#) namespace.

These locations are set on the local drive:

- \Data\SharedData\Enterprise\Persistent
- \Data\SharedData\Enterprise\Non-Persistent

Scenarios

Enterprise shared storage provides support for the following scenarios.

- You can share data within an instance of an app, between instances of the same app, or even between apps assuming they both have the appropriate capability and certificate.
- You can store data on the local hard drive in the \Data\SharedData\Enterprise\Persistent folder and it persists even after the device has been reset.
- Manipulate files, including read, write, and delete of files on a device via Mobile Device Management (MDM) service.

Access enterprise shared storage

The following example shows how to declare the capability to access enterprise shared storage in the package manifest, and how to access the shared storage folders by using the `Windows.Storage.StorageFolder` class.

In your app package manifest, include the following capability:

XML

```
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:mp="http://schemas.microsoft.com/appx/2014/phone/manifest"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
```

```

xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows1
0/restrictedcapabilities"
    IgnorableNamespaces="uap mp rescap">

...

<Capabilities>
    <rescap:Capability Name="enterpriseDeviceLockdown"/>
</Capabilities>

```

To access the shared data location, your app would use the following code.

C#

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using Windows.Storage;

...

// Get the Enterprise Shared Storage folder.
var enterprisePersistentFolderRoot =
@"C:\Data\SharedData\Enterprise\Persistent";

StorageFolder folder =
    await
StorageFolder.GetFolderFromPathAsync(enterprisePersistentFolderRoot);

// Get the files in the folder.
IReadOnlyList<StorageFile> sortedItems =
    await folder.GetFilesAsync();

// Iterate over the results and print the list of files
// to the Visual Studio Output window.
foreach (StorageFile file in sortedItems)
    Debug.WriteLine(file.Name + ", " + file.DateCreated);

```

Tutorial: Create a customer database application

Article • 10/20/2022

This tutorial creates a simple app for managing a list of customers. In doing so, it introduces a selection of basic concepts for enterprise apps in UWP. You'll learn how to:

- Implement Create, Read, Update, and Delete operations against a local SQL database.
- Add a data grid, to display and edit customer data in your UI.
- Arrange UI elements together in a basic form layout.

The starting point for this tutorial is a single-page app with minimal UI and functionality, based on a simplified version of the [Customer Orders Database sample app](#). It's written in C# and XAML, and we're expecting that you've got a basic familiarity with both those languages.

CustomerDatabaseTutorial.App

+	🗑️	🔄	⋮	
FirstName	LastName	Address	Company	
Xavier	Smith	1, rue Pierre-Demoulin, Paris WA 75007	AdventureWorks	
Linda	Gill	Auf Der Steige 100, Dresden WA 01071	Northwind	
Kaylee	Gonzalez	6977 Evergreen Ct., Gold Coast WA 4217	Northwind	
Cesar	Rana	2669 Clark Creek Rd., London WA W1V 5RN	AdventureWorks	
Bryan	Gray	3852 Northridge Dr., Colma WA 94014	Contoso	
Bianca	Hu	2928 Antone Court, Ballard WA 98107	AdventureWorks	
Megan	Torres	9693 Mellowood Street, National City WA 91950	Northwind	
Walter	Suarez	6155 Buckthorn Court, Bellingham WA 98225	Northwind	
Robert	Hill	3830 York Dr., Newport Beach WA 92625	AdventureWorks	
Shaun	Sharma	Unter Linden 924, Kassel WA 34117	Contoso	
Brad	Raje	2585 San Vincente Drive, Gateshead WA GA10	Northwind	
Willie	Raje	5, rue de Courtaboeuf, Paris WA 75017	Contoso	
Ramon	Liu	3989 Terrace Drive, Bellingham WA 98225	Northwind	
Brianna	Williams	5790 Ponderosa Drive, Lebanon WA 97355	Northwind	

Prerequisites

- [Ensure you have the latest version of Visual Studio and the Windows SDK](#)
- [Clone or download the Customer Database Tutorial sample](#)

After you've cloned/downloaded the repo, you can edit the project by opening `CustomerDatabaseTutorial.sln` with Visual Studio.

ⓘ Note

This tutorial is based on the [Customer Orders Database sample](#) which has recently been updated to make use of WinUI and the Windows App SDK. Until this tutorial and code are updated, there will be differences between the two samples.

Part 1: Code of Interest

If you run your app immediately after opening it, you'll see a few buttons at the top of a blank screen. Though it's not visible to you, the app already includes a local SQLite database provisioned with a few test customers. From here, you'll start by implementing a UI control to display those customers, and then move on to adding in operations against the database. Before you begin, here's where you'll be working.

Views

CustomerListPage.xaml is the app's View, which defines the UI for the single page in this tutorial. Any time you need to add or change a visual element in the UI, you'll do it in this file. This tutorial will walk you through adding these elements:

- A **RadDataGrid** for displaying and editing your customers.
- A **StackPanel** to set the initial values for a new customer.

ViewModels

ViewModels\CustomerListPageViewModel.cs is where the fundamental logic of the app is located. Every user action taken in the view will be passed into this file for processing. In this tutorial, you'll add some new code, and implement the following methods:

- **CreateNewCustomerAsync**, which initializes a new **CustomerViewModel** object.
- **DeleteNewCustomerAsync**, which removes a new customer before it's displayed in the UI.
- **DeleteAndUpdateAsync**, which handles the delete button's logic.
- **GetCustomerListAsync**, which retrieves a list of customers from the database.
- **SaveInitialChangesAsync**, which adds a new customer's information to the database.
- **UpdateCustomersAsync**, which refreshes the UI to reflect any customers added or deleted.

CustomerViewModel is a wrapper for a customer's information, which tracks whether or not it's been recently modified. You won't need to add anything to this class, but some of the code you'll add elsewhere will reference it.

For more information on how the sample is constructed, check out the [app structure overview](#).

Part 2: Add the DataGrid

Before you begin to operate on customer data, you'll need to add a UI control to display those customers. To do this, we'll be using a pre-made third-party **RadDataGrid** control. The **Telerik.UI.for.UniversalWindowsPlatform** NuGet package has already been included in this project. Let's add the grid to our project.

1. Open **Views\CustomerListPage.xaml** from the Solution Explorer. Add the following line of code within the **Page** tag to declare a mapping to the Telerik namespace containing the data grid.

XAML

```
xmlns:telerikGrid="using:Telerik.UI.Xaml.Controls.Grid"
```

2. Below the **CommandBar** within the main **RelativePanel** of the View, add a **RadDataGrid** control, with some basic configuration options:

XAML

```
<Grid
  x:Name="CustomerListRoot"
  Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
  <RelativePanel>
    <CommandBar
      x:Name="mainCommandBar"
      HorizontalAlignment="Stretch"
      Background="AliceBlue">
      <!--CommandBar content-->
    </CommandBar>
    <telerikGrid:RadDataGrid
      x:Name="DataGrid"
      BorderThickness="0"
      ColumnDataOperationsMode="Flyout"
      GridLinesVisibility="None"
      GroupPanelPosition="Left"
      RelativePanel.AlignLeftWithPanel="True"
      RelativePanel.AlignRightWithPanel="True"
      RelativePanel.Below="mainCommandBar" />
```

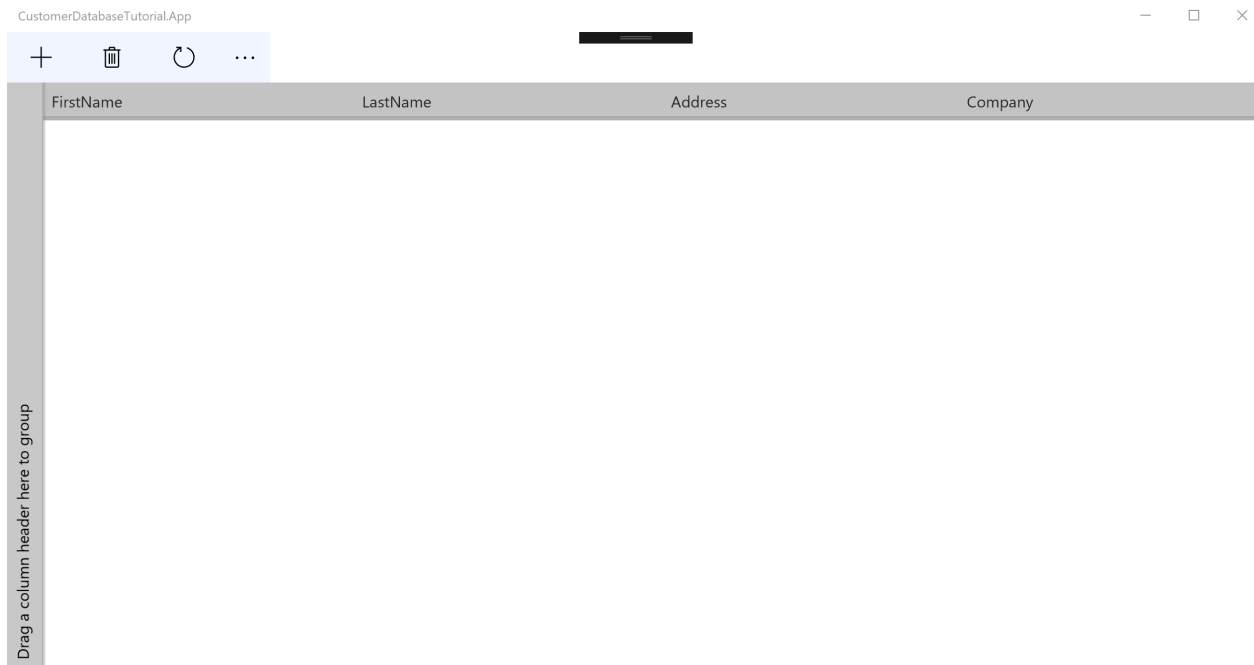
```
</RelativePanel>
</Grid>
```

3. You've added the data grid, but it needs data to display. Add the following lines of code to it:

XAML

```
ItemsSource="{x:Bind ViewModel.Customers}"
UserEditMode="Inline"
```

Now that you have defined a source of data to display, **RadDataGrid** will handle most of the UI logic for you. However, if you run your project, you still won't see any data on display. That's because the ViewModel isn't loading it yet.



Part 3: Read customers

When it's initialized, **ViewModels\CustomerListPageViewModel.cs** calls the **GetCustomerListAsync** method. That method needs to retrieve the test Customer data from the SQLite database that's included in the tutorial.

1. In **ViewModels\CustomerListPageViewModel.cs**, update your **GetCustomerListAsync** method with this code:

```
C#

public async Task GetCustomerListAsync()
{
    var customers = await App.Repository.Customers.GetAsync();
}
```

```

if (customers == null)
{
    return;
}
await DispatcherHelper.ExecuteOnUIThreadAsync(() =>
{
    Customers.Clear();
    foreach (var c in customers)
    {
        Customers.Add(new CustomerViewModel(c));
    }
});
}

```

The **GetCustomerListAsync** method is called when the ViewModel is loaded, but before this step, it didn't do anything. Here, we've added a call to the **GetAsync** method in **Repository/SqlCustomerRepository**. This allows it to contact the repository to retrieve an enumerable collection of Customer objects. It then parses them into individual objects, before adding them to its internal **ObservableCollection** so they can be displayed and edited.

2. Run your app - you'll now see the data grid displaying the list of customers.

CustomerDatabaseTutorial.App

+	🗑	🔄	⋮	
FirstName	LastName	Address	Company	
Xavier	Smith	1, rue Pierre-Demoulin, Paris WA 75007	AdventureWorks	
Linda	Gill	Auf Der Steige 100, Dresden WA 01071	Northwind	
Kaylee	Gonzalez	6977 Evergreen Ct., Gold Coast WA 4217	Northwind	
Cesar	Rana	2669 Clark Creek Rd., London WA W1V 5RN	AdventureWorks	
Bryan	Gray	3852 Northridge Dr., Colma WA 94014	Contoso	
Bianca	Hu	2928 Antone Court, Ballard WA 98107	AdventureWorks	

Part 4: Edit customers

You can edit the entries in the data grid by double-clicking them, but you need to ensure that any changes you make in the UI are also made to your collection of customers in the code-behind. This means you'll have to implement two-way data binding. If you want more information about this, check out our [introduction to data binding](#).

1. First, declare that **ViewModels\CustomerListPageViewModel.cs** implements the **INotifyPropertyChanged** interface:

C#

```
public class CustomerListPageViewModel : INotifyPropertyChanged
```

2. Then, within the main body of the class, add the following event and method:

C#

```
public event PropertyChangedEventHandler PropertyChanged;

public void OnPropertyChanged([CallerMemberName] string propertyName =
null) =>
    PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
```

The **OnPropertyChanged** method makes it easy for your setters to raise the **PropertyChanged** event, which is necessary for two-way data binding.

3. Update the setter for **SelectedCustomer** with this function call:

C#

```
public CustomerViewModel SelectedCustomer
{
    get => _selectedCustomer;
    set
    {
        if (_selectedCustomer != value)
        {
            _selectedCustomer = value;
            OnPropertyChanged();
        }
    }
}
```

4. In **Views\CustomerListPage.xaml**, add the **SelectedCustomer** property to your data grid.

XAML

```
SelectedItem="{x:Bind ViewModel.SelectedCustomer, Mode=TwoWay}"
```

This associates the user's selection in the data grid with the corresponding Customer object in the code-behind. The TwoWay binding mode allows the changes made in the UI to be reflected on that object.

5. Run your app. You can now see the customers displayed in the grid, and make changes to the underlying data through your UI.

CustomerDatabaseTutorial.App

FirstName	LastName	Address	Company
Linda	Gill	Auf Der Steige 100, Dresden WA 01071	Northwind
Kaylee	Gonzalez	6977 Evergreen Ct., Gold Coast WA 4217	Northwind
Cesar	Rana	2669 Clark Creek Rd., London WA W1V 5RN	AdventureWorks
Bryan	Gray	3852 Northridge Dr., Colma WA 94014	Contoso

Part 5: Update customers

Now that you can see and edit your customers, you'll need to be able to push your changes to the database, and to pull any updates that have been made by others.

1. Return to **ViewModels\CustomerListPageViewModel.cs**, and navigate to the **UpdateCustomersAsync** method. Update it with this code, to push changes to the database and to retrieve any new information:

```
C#

public async Task UpdateCustomersAsync()
{
    foreach (var modifiedCustomer in Customers
        .Where(x => x.IsModified).Select(x => x.Model))
    {
        await App.Repository.Customers.UpsertAsync(modifiedCustomer);
    }
    await GetCustomerListAsync();
}
```

This code utilizes the **IsModified** property of **ViewModels\CustomerViewModel.cs**, which is automatically updated whenever the customer is changed. This allows you to avoid unnecessary calls, and to only push changes from updated customers to the database.

Part 6: Create a new customer

Adding a new customer presents a challenge, as the customer will appear as a blank row if you add it to the UI before providing values for its properties. That's not a problem, but here we'll make it easier to set a customer's initial values. In this tutorial, we'll add a simple collapsible panel, but if you had more information to add you could create a separate page for this purpose.

Update the code-behind

1. Add a new private field and public property to **ViewModels\CustomerListPageViewModel.cs**. This will be used to control whether or not the panel is visible.

```
C#

private bool _addingNewCustomer = false;

public bool AddingNewCustomer
{
    get => _addingNewCustomer;
    set
    {
        if (_addingNewCustomer != value)
        {
            _addingNewCustomer = value;
            OnPropertyChanged();
        }
    }
}
```

2. Add a new public property to the ViewModel, an inverse of the value of **AddingNewCustomer**. This will be used to disable the regular command bar buttons when the panel is visible.

```
C#

public bool EnableCommandBar => !AddingNewCustomer;
```

You'll now need a way to display the collapsible panel, and to create a customer to edit within it.

3. Add a new private field and public property to the ViewModel, to hold the newly created customer.

```
C#

private CustomerViewModel _newCustomer;

public CustomerViewModel NewCustomer
{
    get => _newCustomer;
    set
    {
        if (_newCustomer != value)
        {
            _newCustomer = value;
            OnPropertyChanged();
        }
    }
}
```

```
}  
}
```

4. Update your **CreateNewCustomerAsync** method to create a new customer, add it to the repository, and set it as the selected customer:

C#

```
public async Task CreateNewCustomerAsync()  
{  
    CustomerViewModel newCustomer = new CustomerViewModel(new  
Models.Customer());  
    NewCustomer = newCustomer;  
    await App.Repository.Customers.UpsertAsync(NewCustomer.Model);  
    AddingNewCustomer = true;  
}
```

5. Update the **SaveInitialChangesAsync** method to add a newly-created customer to the repository, update the UI, and close the panel.

C#

```
public async Task SaveInitialChangesAsync()  
{  
    await App.Repository.Customers.UpsertAsync(NewCustomer.Model);  
    await UpdateCustomersAsync();  
    AddingNewCustomer = false;  
}
```

6. Add the following line of code as the final line in the setter for **AddingNewCustomer**:

C#

```
OnPropertyChanged(nameof(EnableCommandBar));
```

This will ensure that **EnableCommandBar** is automatically updated whenever **AddingNewCustomer** is changed.

Update the UI

1. Navigate back to **Views\CustomerListPage.xaml**, and add a **StackPanel** with the following properties between your **CommandBar** and your data grid:

XAML

```

<StackPanel
    x:Name="newCustomerPanel"
    Orientation="Horizontal"
    x:Load="{x:Bind ViewModel.AddingNewCustomer, Mode=OneWay}"
    RelativePanel.Below="mainCommandBar">
</StackPanel>

```

The **x:Load** attribute ensures that this panel only appears when you're adding a new customer.

2. Make the following change to the position of your data grid, to ensure that it moves down when the new panel appears:

XAML

```
RelativePanel.Below="newCustomerPanel"
```

3. Update your stack panel with four **TextBox** controls. They'll bind to the individual properties of the new customer, and allow you to edit its values before you add it to the data grid.

XAML

```

<StackPanel
    x:Name="newCustomerPanel"
    Orientation="Horizontal"
    x:Load="{x:Bind ViewModel.AddingNewCustomer, Mode=OneWay}"
    RelativePanel.Below="mainCommandBar">
    <TextBox
        Header="First name"
        PlaceholderText="First"
        Margin="8,8,16,8"
        MinWidth="120"
        Text="{x:Bind ViewModel.NewCustomer.FirstName, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>
    <TextBox
        Header="Last name"
        PlaceholderText="Last"
        Margin="0,8,16,8"
        MinWidth="120"
        Text="{x:Bind ViewModel.NewCustomer.LastName, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>
    <TextBox
        Header="Address"
        PlaceholderText="1234 Address St, Redmond WA 00000"
        Margin="0,8,16,8"
        MinWidth="280"
        Text="{x:Bind ViewModel.NewCustomer.Address, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>

```



```

<TextBox
    Header="Company"
    PlaceholderText="Company"
    Margin="0,8,16,8"
    MinWidth="120"
    Text="{x:Bind ViewModel.NewCustomer.Company, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>
</StackPanel>

```

4. Add a simple button to your new stack panel to save the newly-created customer:

XAML

```

<StackPanel>
    <!--Text boxes from step 3-->
    <AppBarButton
        x:Name="SaveNewCustomer"
        Click="{x:Bind ViewModel.SaveInitialChangesAsync}"
        Icon="Save"/>
</StackPanel>

```

5. Update the **CommandBar**, so the regular create, delete, and update buttons are disabled when the stack panel is visible:

XAML

```

<CommandBar
    x:Name="mainCommandBar"
    HorizontalAlignment="Stretch"
    IsEnabled="{x:Bind ViewModel.EnableCommandBar, Mode=OneWay}"
    Background="AliceBlue">
    <!--App bar buttons-->
</CommandBar>

```

6. Run your app. You can now create a customer and input its data in the stack panel.

CustomerDatabaseTutorial.App

First name:
 Last name:
 Address:
 Company:

FirstName	LastName	Address	Company
Xavier	Smith	1, rue Pierre-Demoulin, Paris WA 75007	AdventureWorks
Linda	Gill	Auf Der Steige 100, Dresden WA 01071	Northwind
Kaylee	Gonzalez	6977 Evergreen Ct., Gold Coast WA 4217	Northwind
Cesar	Rana	2669 Clark Creek Rd., London WA W1V 5RN	AdventureWorks

Part 7: Delete a customer

Deleting a customer is the final basic operation that you need to implement. When you delete a customer you've selected within the data grid, you'll want to immediately call **UpdateCustomersAsync** in order to update the UI. However, you don't need to call that method if you're deleting a customer you've just created.

1. Navigate to **ViewModels\CustomerListPageViewModel.cs**, and update the **DeleteAndUpdateAsync** method:

```
C#

public async void DeleteAndUpdateAsync()
{
    if (SelectedCustomer != null)
    {
        await
App.Repository.Customers.DeleteAsync(_selectedCustomer.Model.Id);
    }
    await UpdateCustomersAsync();
}
```

2. In **Views\CustomerListPage.xaml**, update the stack panel for adding a new customer so it contains a second button:

```
XAML

<StackPanel>
    <!--Text boxes for adding a new customer-->
    <AppBarButton
        x:Name="DeleteNewCustomer"
        Click="{x:Bind ViewModel.DeleteNewCustomerAsync}"
        Icon="Cancel"/>
    <AppBarButton
        x:Name="SaveNewCustomer"
        Click="{x:Bind ViewModel.SaveInitialChangesAsync}"
        Icon="Save"/>
</StackPanel>
```

3. In **ViewModels\CustomerListPageViewModel.cs**, update the **DeleteNewCustomerAsync** method to delete the new customer:

```
C#

public async Task DeleteNewCustomerAsync()
{
    if (NewCustomer != null)
    {
        await
App.Repository.Customers.DeleteAsync(_newCustomer.Model.Id);
        AddingNewCustomer = false;
    }
}
```

```
}  
}
```

4. Run your app. You can now delete customers, either within the data grid or in the stack panel.

The screenshot shows the 'CustomerDatabaseTutorial.App' interface. At the top, there's a toolbar with icons for adding (+), deleting (trash), refreshing (circular arrow), and a menu (three dots). Below the toolbar is a form with four input fields: 'First name' (containing 'First'), 'Last name' (containing 'Last'), 'Address' (containing '1234 Address St, Redmond WA 00000'), and 'Company' (containing 'Company'). To the right of the form are a close button (X) and a save button (floppy disk). Below the form is a data grid with four columns: 'FirstName', 'LastName', 'Address', and 'Company'. The grid contains three rows of customer data.

FirstName	LastName	Address	Company
Xavier	Smith	1, rue Pierre-Demoulin, Paris WA 75007	AdventureWorks
Linda	Gill	Auf Der Steige 100, Dresden WA 01071	Northwind
Kaylee	Gonzalez	6977 Evergreen Ct., Gold Coast WA 4217	Northwind

Conclusion

Congratulations! With all this done, your app now has a full range of local database operations. You can create, read, update, and delete customers within your UI, and these changes are saved to your database and will persist across different launches of your app.

Now that you're finished, consider the following:

- If you haven't already, check out the [app structure overview](#) for more information on why the app is built how it is.
- Explore the [full Customer Orders Database sample](#) to see the app this tutorial was based on.

Or if you're up for a challenge, you can continue onwards...

Going further: Connect to a remote database

We've provided a step-by-step walkthrough of how to implement these calls against a local SQLite database. But what if you want to use a remote database, instead?

If you want to give this a try, you'll need your own Azure Active Directory (AAD) account and the ability to host your own data source.

You'll need to add authentication, functions to handle REST calls, and then create a remote database to interact with. There's code in the full [Customer Orders Database sample](#) that you can reference for each necessary operation.

Settings and configuration

The necessary steps to connect to your own remote database are spelled out in the [sample's readme](#). You'll need to do the following:

- Provide your Azure account client Id to [Constants.cs](#).
- Provide the url of the remote database to [Constants.cs](#).
- Provide the connection string for the database to [Constants.cs](#).
- Associate your app with the Microsoft Store.
- Copy over the [Service project](#) into your app, and deploy it to Azure.

Authentication

You'll need to create a button to start an authentication sequence, and a popup or a separate page to gather a user's information. Once you've created that, you'll need to provide code that requests a user's information and uses it to acquire an access token. The Customer Orders Database sample wraps Microsoft Graph calls with the **WebAccountManager** library to acquire a token and handle the authentication to an AAD account.

- The authentication logic is implemented in [AuthenticationViewModel.cs](#).
- The authentication process is displayed in the custom [AuthenticationControl.xaml](#) control.

REST calls

You won't need to modify any of the code we added in this tutorial in order to implement REST calls. Instead, you'll need to do the following:

- Create new implementations of the **ICustomerRepository** and **ITutorialRepository** interfaces, implementing the same set of functions through REST instead of SQLite. You'll need to serialize and deserialize JSON, and can wrap your REST calls in a separate **HttpHelper** class if you need to. Refer to [the full sample](#) for specifics.
- In **App.xaml.cs**, create a new function to initialize the REST repository, and call it instead of **SqliteDatabase** when the app is initialized. Again, refer to [the full sample](#).

Once all three of these steps are complete, you should be able to authenticate to your AAD account through your app. REST calls to the remote database will replace the local SQLite calls, but the user experience should be the same. If you're feeling even more ambitious, you can add a settings page to allow the user to dynamically switch between the two.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Customer database app structure

Article • 10/20/2022

Complex line-of-business apps often have many pages and features and a great many lines of code. Because of this, it's of vital importance that you design your app around a predictable structure. There are several application design patterns which are suitable for enterprise apps, but they are all built around the goal of making a large-scale application easier to understand and work with.

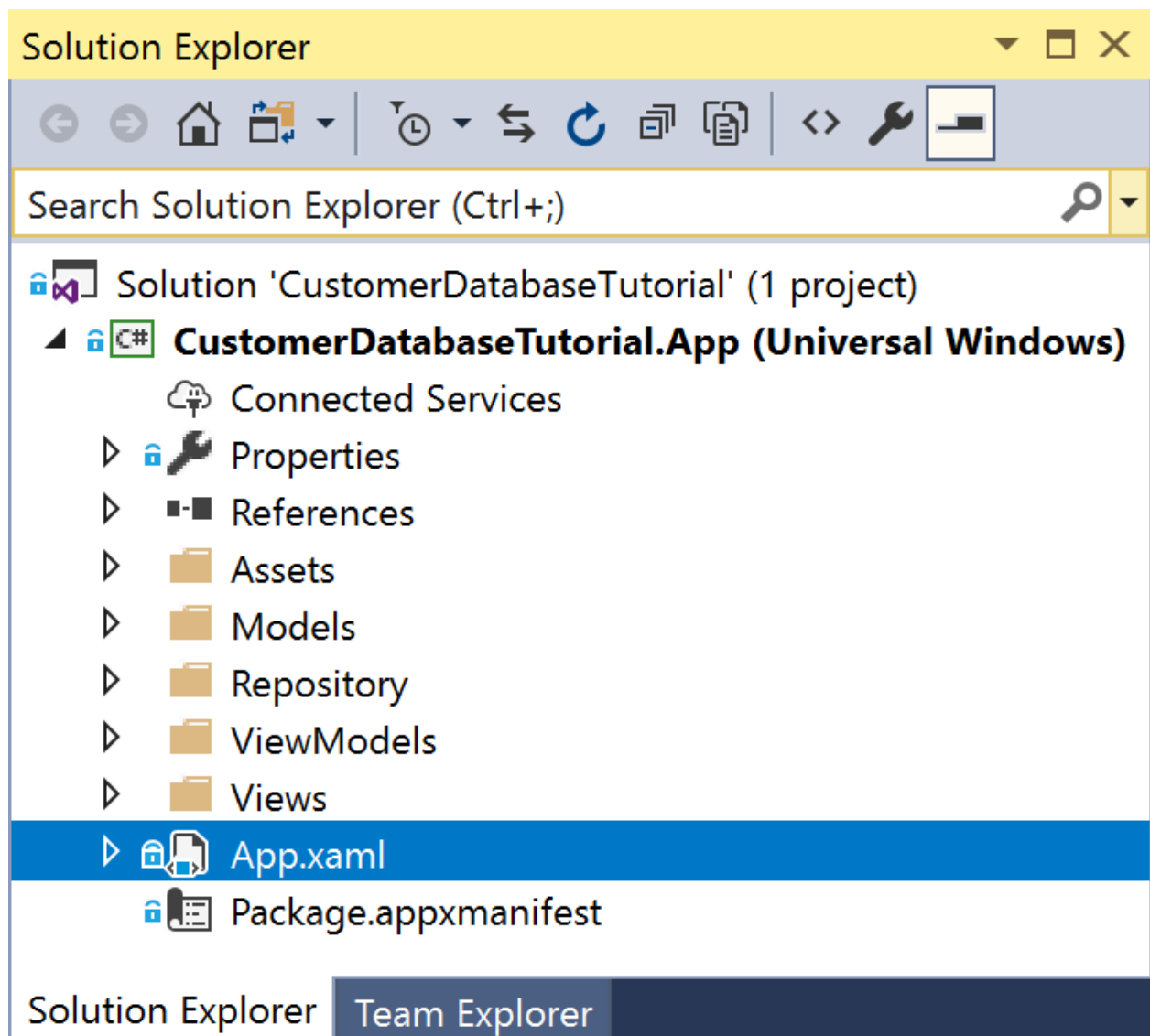
While the [Customer database tutorial](#) presents a single-page app for simplicity's sake, it implements the Model-View-ViewModel (MVVM) app design pattern to showcase these ideas in action. As the name implies, the MVVM design pattern separates the core app logic into three categories:

- Models are the classes that contain the application's data.
- Views are the UI of any given page.
- ViewModels provide the application logic. This can include handling user actions from the View and/or managing interactions with the Models.

While this app isn't a perfect and archetypical example of MVVM, it does show the main principles of separation of concerns in action. [Check out the app here.](#) ↗

Application structure

After you open the app, start by examining the **Solution Explorer**. Some of what you see there should be familiar if you've worked with a UWP app before, but you'll also see a collection of folders that hold the app's component parts.



Views

All the app's UI is defined in the Views folder. Because our tutorial is a single-page app right now, this means there's only one view - **CustomerListPage**. It has both XAML UI markup and xaml.cs code-behind - these two files make up one View. You'll be adding UI elements to **CustomerListPage.xaml**.

ⓘ Note

You might notice that this app doesn't have a MainPage. That's because we specify in **App.xaml.cs** that the app should launch **CustomerListPage** upon launching.

ViewModels

Though this app only has one View, it has two ViewModels. Why is this?

CustomerListPageViewModel.cs is a standard ViewModel in the MVVM pattern. It's where the fundamental logic of the app's page is located, and the page you'll be working with the most in the tutorial. Every UI action taken by the user is passed through the View to this ViewModel for processing.

CustomerViewModel.cs, however, isn't associated with any specific View. Instead, it associates a programmatic concept (which properties have been edited) with the data contained in an individual customer's Model.

Models

This app contains three Models, which store the app's data and provide interfaces for interacting with the repository. While these are critical parts of the app, they aren't something that you'll be directly editing in the tutorial.

Most important is **Customer.cs**, which describes the Customer data structure which you'll be using in the tutorial.

ⓘ Note

The tutorial ignores the *Email* and *Phone* properties of the Customer object. If you want to go beyond what's presented here, adding these two properties into the UI of the app is a good first step.

Repository

The Repository folder contains classes that construct and interact with the local SQLite database. For the tutorial, the SQLite database is presented as-is. While you'll be adding in code to **CustomerListPageViewModel.cs** to call methods defined by these classes, you won't need to make any changes in order to set them up.

For more information on SQLite in UWP, [see this article](#).

If you try the "Going further" section of the tutorial, this is where you'll create a class to connect to the remote REST database. It will also implement the **ICustomerRepository** interface defined in the Models section, but it will look very different than its SQLite counterpart.

Other elements

As is usual for UWP apps, the application launch behavior is defined in the **App.xaml.cs** class. Most of the code here is the default code for any UWP app. But we've already made a few small changes:

- We've specified that the app should display **CustomerListPage** on launch.
- We've created a Repository object, which will hold the data source we're using.
- We've added a **SQLiteDatabase** method, which initializes the local database and sets it as the specified Repository.

If you try the "Going further" section, you'll add a similar method to initialize a REST Repository object. Because we've separated our concerns and are using the same defined interface for both SQLite and REST operations, this will be the only existing code you'll need to change to use REST instead of SQLite in your app.

Next steps

If you've already completed the tutorial, then you can check out the [full sample app](#) to see how these features are implemented on a larger scale.

Otherwise, now that you know why everything is where it is, you should [return to the tutorial](#) and work with the structure that we've just covered.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Overview of Progressive Web Apps (PWAs)

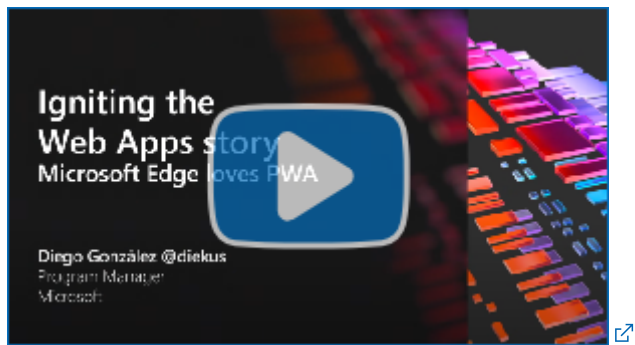
Article • 01/24/2023

Progressive Web Apps (PWAs) are applications that you build by using web technologies, and that can be installed and can run on all devices, from one codebase.

PWAs provide native-like experiences to your users on supporting devices. They adapt to the capabilities supported by each device and they can also run in web browsers, like websites.

To start building a PWA, see [Get started with PWAs](#).

Video: Ignite | March 2021 | Igniting the Web Apps Story



PWA benefits

Native-like experiences

When installed on a device, PWAs function just like other apps. For example:

- PWAs have their own application icons that can be added to a device's home screen or task bar.
- PWAs can be launched automatically when an associated file type is opened.
- PWAs can run when the user signs in.
- PWAs can be submitted to application stores, such as the Microsoft Store.

Advanced capabilities

PWAs also have access to advanced capabilities. For example:

- PWAs can continue working when the device is offline.

- PWAs support push notifications.
- PWAs can perform periodic updates even when the application is not running.
- PWAs can access hardware features.

Web-related advantages

Finally, PWAs can run in web browsers, just like websites. Running like websites gives PWAs with advantages, such as:

- PWAs can be indexed by search engines.
- PWAs can be shared and launched from a standard web link.
- PWAs are safe for users because they use secure HTTPS endpoints and other user safeguards.
- PWAs adapt to the user's screen size or orientation, and input method.
- PWAs can use advanced web APIs such as WebBluetooth, WebUSB, WebPayment, WebAuthn, or WebAssembly.

Lower development cost

PWAs have a **much lower cross-platform development cost** than compiled apps that require a specific, separate codebase for each platform, such as Android, iOS, and each desktop operating system.

With a PWA, you can use a single codebase that's shared between your website, mobile app, and desktop app (across operating systems).

Cross-device compatibility

Progressive Web Apps (PWAs) are built using HTML, CSS, and JavaScript code that's hosted on web servers and that runs in web browser engines. They can either be used directly in a web browser, like a website, or installed on a device from an app store, or by using the app installation feature of a supporting browser. To learn more about installing PWAs in Microsoft Edge, see [Installing a PWA](#).

The user experience of a PWA scales up or down with device capabilities. In particular, on Windows with Microsoft Edge, you can build immersive experiences that deeply integrate with the operating system. Using only web technologies ensures that your app will work on other browsers, operating systems, and devices.

Desktop computing continues to be the primary productivity environment for many users. PWAs can provide fully tailored, installable experiences on a desktop operating

system that inherit the benefits of that environment. Yet PWAs can also be installed on mobile devices, or used directly in web browsers.

Bridging the gap between web and native

There isn't much you can't do today with web technologies. Many features that were once only possible by using device-specific languages can now be achieved by using standard web technologies. For example:

- Handle files.
- Share content with other apps.
- Access the clipboard.
- Sync data and fetch resources in the background.
- Access device hardware such as Bluetooth and USB.
- Store content in databases.
- Take advantage of hardware accelerated graphics.
- Use CSS layouts, animations, and filters to create advanced designs.
- Run near-compiled performance code with WebAssembly.

Thanks to Microsoft Edge, PWAs on desktop can take full advantage of these capabilities to deliver what users expect desktop apps to be able to do. These capabilities provide more engaging experiences across web and desktop platforms.

To find examples of what PWAs can do, see [Myth Busting PWAs](#).

The Microsoft Store

Because Progressive Web Apps (PWAs) are just like other apps in the [Microsoft Store](#), users can fully engage with them—from discovery, to installation, to execution—without ever opening the browser.

The Microsoft Store provides a trustworthy and familiar experience for users to install your app. Additionally, you can view detailed usage statistics and charts that let you know how your apps in the Microsoft Store are doing.

See [Publish a PWA to the Microsoft Store](#).

Success stories

Using Progressive Web App (PWA) technologies is a great way to make your app **safe**, **discoverable**, **linkable**, **easy to install** and update, **responsive**, and **network**

independent. Many businesses have had success with PWAs. For example:

- The Starbucks PWA has increased daily active users twice. Orders on desktop are nearly the same rate as mobile ([source](#)).
- Trivago saw an increase of 150% for people who add its PWA to the home screen. The increased engagement led to a 97% increase in clockouts to hotel offers ([source](#)).
- Tinder cut load times from 11.91 seconds to 4.68 seconds with their PWA. The app is 90% smaller than the compiled Android app ([source](#)).

Read more success stories on the [PWA Stats](#) website.

See also

- [Progressive Web Apps](#) at MDN Web Docs.
- [Progressive Web Apps](#) at web.dev.
- [Progressive Web Apps](#) at Wikipedia.
- [PWA Q&A](#)

Feedback

Was this page helpful?

 Yes

 No

Audio, video, and camera

Article • 10/20/2022

This section provides information about creating Universal Windows Platform (UWP) apps that capture, play back, or edit photos, videos, or audio.

Topic	Description
Camera	Lists the camera features that are available for UWP apps, and links to the how-to articles that show how to use them.
Media playback	Provides information about creating UWP apps that use audio and video playback.
Detect faces in images or videos	Shows you how to use the FaceTracker for tracking faces over time in a sequence of video frames.
Media compositions and editing	Shows you how to use the APIs in the Windows.Media.Editing namespace to quickly develop apps that enable the users to create media compositions from audio and video source files.
Custom video effects	Describes how to create a Windows Runtime component that implements the IBasicVideoEffect interface to allow you to create custom effects for video streams.
Custom audio effects	Describes how to create a Windows Runtime component that implements the IBasicAudioEffect interface to allow you to create custom effects for audio streams.
Create, edit, and save bitmap images	Explains how to load and save image files by using the SoftwareBitmap object to represent bitmap images.
Audio device information properties	Lists the device information properties related to audio devices.
Detect and respond to audio state changes	Explains how UWP apps can detect and respond to system-initiated changes in audio stream levels.
Transcode media files	Shows you how to use the Windows.Media.Transcoding APIs to transcode video files from one format to another.
Process media files in the background	Shows you how to use the MediaProcessingTrigger and a background task to process media files in the background.

Topic	Description
Audio graphs	Shows you how to use the APIs in the Windows.Media.Audio namespace to create audio graphs for audio routing, mixing, and processing scenarios.
MIDI	Shows you how to enumerate MIDI (Musical Instrument Digital Interface) devices and send and receive MIDI messages from a UWP app.
Import media from a device	Describes how to import media from a device, including searching for available media sources, importing files such as videos, photos, and sidecar files, and deleting the imported files from the source device.
Camera-independent Flashlight	Shows you how to access and use a device's lamp, if one is present. Lamp functionality is managed separately from the device's camera and camera flash functionality.
Supported codecs	Lists the audio, video, and image codec and format support for UWP apps.
Query for installed codecs	Shows you how to query for audio and video encoders and decoders that are installed on a device.
Screen capture	Describes how to use the Windows.Graphics.Capture namespace to acquire frames from a display or application window, to create video streams or snapshots to build collaborative and interactive experiences.

See also

- [Develop UWP apps](#)

Camera

Article • 10/20/2022

This section provides guidance for creating Universal Windows Platform (UWP) apps that use the camera or microphone to capture photos, video, or audio.

Use the Windows built-in camera UI

 Expand table

Topic	Description
Capture photos and video with Windows built-in camera UI	Shows how to use the CameraCaptureUI class to capture photos or videos using the camera UI built into Windows. If you simply want to enable the user to capture a photo or video and return the result to your app, this is the quickest and easiest way to do it.

Basic MediaCapture tasks

 Expand table

Topic	Description
Display the camera preview	Shows how to quickly display the camera preview stream within a XAML page in a UWP app.
Basic photo, video, and audio capture with MediaCapture	Shows the simplest way to capture photos and video using the MediaCapture class. The MediaCapture class exposes a robust set of APIs that provide low-level control over the capture pipeline and enable advanced capture scenarios, but this article is intended to help you add basic media capture to your app quickly and easily.
Camera UI features for mobile devices	Shows you how to take advantage of special camera UI features that are only present on mobile devices.

Advanced MediaCapture tasks

 Expand table

Topic	Description
Handle device and screen orientation with MediaCapture	Shows you how to handle device orientation when capturing photos and videos by using a helper class.
Discover and select camera capabilities with camera profiles	Shows how to use camera profiles to discover and manage the capabilities of different video capture devices. This includes tasks such as selecting profiles that support specific resolutions or frame rates, profiles that support simultaneous access to multiple cameras, and profiles that support HDR.
Set format, resolution, and frame rate for MediaCapture	Shows you how to use the IMediaEncodingProperties interface to set the resolution and frame rate of the camera preview stream and captured photos and video. It also shows how to ensure that the aspect ratio of the preview stream matches that of the captured media.
HDR and low-light photo capture	Shows you how to use the AdvancedPhotoCapture class to capture High Dynamic Range (HDR) and low-light photos.
Manual camera controls for photo and video capture	Shows you how to use manual device controls to enable enhanced photo and video capture scenarios including optical image stabilization and smooth zoom.
Manual camera controls for video capture	Shows you how to use manual device controls to enable enhanced video capture scenarios including HDR video and exposure priority.
Video stabilization effect for video capture	Shows you how to use the video stabilization effect.
Scene analysis for MediaCapture	Shows you how to use the SceneAnalysisEffect and the FaceDetectionEffect to analyze the content of the media capture preview stream.
Capture a photo sequence with VariablePhotoSequence	Shows you how to capture a variable photo sequence, which allows you to capture multiple frames of images in rapid succession and configure each frame to use different focus, flash, ISO, exposure, and exposure compensation settings.
Process media frames with MediaFrameReader	Shows you how to use a MediaFrameReader with MediaCapture to get media frames from one or more available sources, including color, depth, and infrared cameras, audio devices, or even custom frame sources such as those that produce skeletal tracking frames. This feature is designed to be used by apps that perform real-time processing of media frames, such as augmented reality and depth-aware camera apps.
Get a preview frame	Shows you how to get a single preview frame from the media capture preview stream.

UWP app samples for camera

- [Camera face detection sample](#) ↗
- [Camera preview frame sample](#) ↗
- [Camera HDR sample](#) ↗
- [Camera manual controls sample](#) ↗
- [Camera profile sample](#) ↗
- [Camera resolution sample](#) ↗
- [Camera starter kit](#) ↗
- [Camera video stabilization sample](#) ↗

Related topics

- [Audio, video, and camera](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) ↗ | [Get help at Microsoft Q&A](#)

Capture photos and video with the Windows built-in camera UI

Article • 10/20/2022

This article describes how to use the [CameraCaptureUI](#) class to capture photos or videos by using the camera UI built into Windows. This feature is easy to use. It allows your app to get a user-captured photo or video with just a few lines of code.

If you want to provide your own camera UI, or if your scenario requires more robust, low-level control of the capture operation, then you should use the [MediaCapture](#) class, and implement your own capture experience. For more information, see [Basic photo, video, and audio capture with MediaCapture](#).

ⓘ Note

You shouldn't specify the **webcam** nor **microphone** capabilities in your app manifest file if your app only uses **CameraCaptureUI**. If you do, your app will be displayed in the device's camera privacy settings, but even if the user denies camera access to your app, this won't prevent the **CameraCaptureUI** from capturing media.

This is because the Windows built-in camera app is a trusted first-party app that requires the user to initiate photo, audio, and video capture with a button press. Your app may fail Windows Application Certification Kit certification when submitted to Microsoft Store if you specify the webcam or microphone capabilities when using **CameraCaptureUI** as your only photo capture mechanism.

You must specify the **webcam** or **microphone** capabilities in your app manifest file if you're using **MediaCapture** to capture audio, photos, or video programmatically.

Capture a photo with CameraCaptureUI

To use the camera capture UI, include the [Windows.Media.Capture](#) namespace in your project. To do file operations with the returned image file, include [Windows.Storage](#).

C#

```
using Windows.Media.Capture;  
using Windows.Storage;
```

To capture a photo, create a new **CameraCaptureUI** object. By using the object's **PhotoSettings** property, you can specify properties for the returned photo, such as the image format of the photo. By default, the camera capture UI supports cropping the photo before it's returned. This can be disabled with the **AllowCropping** property. This example sets the **CroppedSizeInPixels** to request that the returned image be 200 x 200 in pixels.

ⓘ Note

Image cropping in the **CameraCaptureUI** isn't supported for devices in the Mobile device family. The value of the **AllowCropping** property is ignored when your app is running on these devices.

Call **CaptureFileAsync** and specify **CameraCaptureUIMode.Photo** to specify that a photo should be captured. The method returns a **StorageFile** instance containing the image if the capture is successful. If the user cancels the capture, the returned object is null.

C#

```
CameraCaptureUI captureUI = new CameraCaptureUI();
captureUI.PhotoSettings.Format = CameraCaptureUIPhotoFormat.Jpeg;
captureUI.PhotoSettings.CroppedSizeInPixels = new Size(200, 200);

StorageFile photo = await
captureUI.CaptureFileAsync(CameraCaptureUIMode.Photo);

if (photo == null)
{
    // User cancelled photo capture
    return;
}
```

The **StorageFile** containing the captured photo is given a dynamically generated name and saved in your app's local folder. To better organize your captured photos, you can move the file to a different folder.

C#

```
StorageFolder destinationFolder =
    await
ApplicationData.Current.LocalFolder.CreateFolderAsync("ProfilePhotoFolder",
    CreationCollisionOption.OpenIfExists);

await photo.CopyAsync(destinationFolder, "ProfilePhoto.jpg",
```

```
NameCollisionOption.ReplaceExisting);  
await photo.DeleteAsync();
```

To use your photo in your app, you may want to create a [SoftwareBitmap](#) object that can be used with several different Universal Windows app features.

First, include the [Windows.Graphics.Imaging](#) namespace in your project.

C#

```
using Windows.Storage.Streams;  
using Windows.Graphics.Imaging;
```

Call [OpenAsync](#) to get a stream from the image file. Call [BitmapDecoder.CreateAsync](#) to get a bitmap decoder for the stream. Then, call [GetSoftwareBitmap](#) to get a [SoftwareBitmap](#) representation of the image.

C#

```
IRandomAccessStream stream = await photo.OpenAsync(FileAccessMode.Read);  
BitmapDecoder decoder = await BitmapDecoder.CreateAsync(stream);  
SoftwareBitmap softwareBitmap = await decoder.GetSoftwareBitmapAsync();
```

To display the image in your UI, declare an [Image](#) control in your XAML page.

XML

```
<Image x:Name="imageControl" Width="200" Height="200"/>
```

XML

```
<Image x:Name="imageControl" Width="200" Height="200"/>
```

To use the software bitmap in your XAML page, include the using [Windows.UI.Xaml.Media.Imaging](#) namespace in your project.

C#

```
using Windows.UI.Xaml.Media.Imaging;
```

The [Image](#) control requires that the image source be in BGRA8 format with premultiplied alpha or no alpha. Call the static method [SoftwareBitmap.Convert](#) to create a new software bitmap with the desired format. Next, create a new